

DeepScan

Plataforma de Monitoramento Marítimo e Alertas de Desastres Naturais

Domain Driven Design Using Java

Global Solution 2026 — FIAP

Equipe

Gustavo Souza Nascimento — RM567134

Enzo Yukio Oyadomari — RM561032

Hugo Souza de Jesus — RM568542

Lucas Campanha dos Santos — RM566815

Lucas Marcelino Pompeu — RM567010

Sao Paulo — 2026

Sumario

1. Objetivo e Escopo do Projeto	4
Principais funcionalidades	4
2. Descricao da Solucao	4
Cenario de uso	4
Frontend integrado	4
3. Arquitetura e Camadas	4
Pacotes (src/main/java/br/com/fiap)	5
4. Tecnologias Utilizadas	5
5. Instrucoes para Executar o Projeto	5
5.1 Pre-requisitos	5
5.2 Variaveis de Ambiente	6
5.3 Importar o projeto	6
5.4 Criar as tabelas no Oracle	6
5.5 Subir o servidor em modo dev	6
5.6 Rodar via Docker (opcional)	6
6. Endpoints da API REST	7
6.1 Estacoes de Monitoramento	7
6.2 Leituras de Telemetria	7
6.3 Alertas	7
6.4 Especies Marinhas	7
6.5 Avistamentos	8
6.6 Zonas de Monitoramento	8
6.7 Exemplo de requisicao	8
7. Regras de Negocio (Camada BO)	8
7.1 EstacaoMonitoraBO	8
7.2 LeituraTelemetriaBO	8
7.3 AlertaBO — Classificacao automatica de risco	9
7.4 EspecieBO	9
7.5 AvistamentoBO	9
7.6 ZonaMonitoraBO	9

8. Tratamento de Excecoes	9
Formato da resposta de erro	9
9. Banco de Dados	10
9.1 Conexao	10
9.2 Tabelas	10
9.3 Geracao de IDs	10
9.4 Script DDL	10
10. CORS	10
11. Deploy	11
11.1 Pipeline	11
11.2 Cadeia de exposicao publica	11
11.3 Imagem Docker	11
12. Links e Recursos	11
13. Apendice — Prints da Aplicacao	12

1. Objetivo e Escopo do Projeto

O DeepScan é um sistema de monitoramento marítimo e de biodiversidade oceânica desenvolvido em Java 17 com o framework Quarkus. O sistema integra dados de telemetria coletados por estações de monitoramento (boias, satélites e estações submarinas) com registros de avistamento de espécies marinhas, gerando alertas automáticos de risco a partir dos dados sensorizados.

O projeto foi desenvolvido como solução para o desafio Global Solution 2026 da FIAP, abordando o tema de preservação dos oceanos e monitoramento de desastres naturais marítimos, como tsunamis e ciclones.

Principais funcionalidades

- Cadastro e gerenciamento de estações de monitoramento marítimo (CRUD completo).
- Registro de leituras de telemetria com dados oceanográficos e sísmicos.
- Geração automática de alertas de risco (ALTO, MEDIO, BAIXO) ao registrar leituras.
- Registro de avistamentos de espécies marinhas vinculados às estações.
- Catálogo de espécies marinhas com classificação IUCN de conservação.
- Zonas de monitoramento geográfico.
- API RESTful integrada com o frontend (Vercel).
- Tratamento de exceções padronizado com códigos HTTP e corpo JSON consistente.

2. Descrição da Solução

O DeepScan modela o ciclo completo de monitoramento marítimo. Uma estação registra leituras periódicas de sensores oceanográficos e sísmicos. Quando os valores ultrapassam limites críticos, o sistema gera automaticamente um alerta categorizado por nível de risco. Operadores podem consultar alertas pendentes, marca-los como resolvidos e registrar avistamentos de espécies na área da estação.

Cenário de uso

Uma boia no Atlântico Sul detecta um terremoto de magnitude 7.5 a 30 km de profundidade. O sistema registra a leitura via POST /deepscan/leituras, o LeituraTelemetriaBO valida os dados, o AlertaBO calcula o risco como ALTO e insere automaticamente um alerta no banco. Paralelamente, um operador registra o avistamento de uma baleia-jubarte na mesma região via POST /deepscan/avistamentos.

Frontend integrado

A API é consumida pelo frontend hospedado em <https://deepscanfiap.vercel.app>. O CORS do backend libera essa origem para todos os verbos REST.

3. Arquitetura e Camadas

O projeto segue uma arquitetura em camadas inspirada em DDD, com responsabilidades claramente separadas. O fluxo de uma requisição percorre as camadas conforme abaixo:

```
Cliente HTTP
-> Resource (JAX-RS, validação de roteamento)
-> BO (validações e regras de negócio)
```

-> DAO (PreparedStatement direto, sem JPA)
-> Oracle FIAP

Excecoes lancadas em qualquer camada sao capturadas por @Provider
ExceptionHandler que padronizam a resposta HTTP.

Pacotes (src/main/java/br/com/fiap)

Pacote	Responsabilidade
entities/	Modelos do dominio. POJOs sem anotacoes ORM.
conexoes/	Fabrica de conexoes JDBC com Oracle.
dao/	Acesso a dados via PreparedStatement. CRUD literal por entidade.
bo/	Validacoes e regras de negocio. Orquestra DAOs e BOs auxiliares.
exceptions/	Excecoes customizadas: DadoInvalido, RecursoNaoEncontrado, Persistencia.
mapper/	@Provider ExceptionMappers que traduzem excecoes em status HTTP + corpo JSON.
filter/	CorsFilter (@Provider) que adiciona cabecalhos CORS em toda resposta.
resource/	Endpoints JAX-RS. Um resource por entidade. Sem try/catch.
main/	Main.java — entry point @QuarkusMain da aplicacao.

4. Tecnologias Utilizadas

Tecnologia	Versao	Finalidade
Java	JDK 17	Linguagem
Quarkus	3.8.3	Framework REST (RESTEasy Reactive + Jackson)
Maven	3.9	Build e gerenciamento de dependencias
Oracle Database	FIAP (oracle.fiap.com.br)	Banco relacional
Oracle JDBC	ojdbc11 23.3	Driver JDBC
Gson	2.10.1	Serializacao auxiliar
Docker	—	Empacotamento da aplicacao
GitHub Actions	—	CI/CD em runner self-hosted
Nginx	—	Proxy reverso na frente do container
Cloudflare Tunnel	—	Exposicao HTTPS publica
IntelliJ IDEA	2025.x	IDE de desenvolvimento
Postman / Insomnia	—	Testes manuais dos endpoints

5. Instrucoes para Executar o Projeto

5.1 Pre-requisitos

- JDK 17 instalado e configurado no PATH.
- Maven 3.9 (ou usar o wrapper se for adicionado ao projeto).

- IntelliJ IDEA ou outra IDE com suporte a projetos Maven.
- Acesso de rede ao banco Oracle da FIAP (oracle.fiap.com.br:1521).
- Postman ou Insomnia para testar os endpoints.
- Docker (opcional) — necessario apenas para construir a imagem ou rodar o container.

5.2 Variaveis de Ambiente

A aplicacao le as credenciais do banco das seguintes variaveis. Se qualquer uma estiver ausente, a aplicacao falha rapido com `IllegalStateException`.

Variavel	Descricao
DB_URL	URL JDBC do banco. Ex.: jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
DB_USER	Usuario do banco (RM do aluno).
DB_PASSWORD	Senha do banco.

Em producao, sao configuradas como GitHub Secrets do repositorio e injetadas no container pelo workflow de deploy.

5.3 Importar o projeto

- Clone o repositorio: `git clone https://github.com/deep-scan-fiap/DeepScan-Backend-Java.git`
- Abra o IntelliJ: *File > Open*, selecione o arquivo `pom.xml`, escolha *Open as Project*.
- Aguarde o painel Maven baixar as dependencias automaticamente.

5.4 Criar as tabelas no Oracle

Execute o script `src/main/resources/create_tables.sql` em sua ferramenta Oracle de preferencia (SQL Developer, DBeaver, ou via `sqlplus`). O script faz `DROP / CREATE` das tabelas e popula com dados de exemplo.

5.5 Subir o servidor em modo dev

```
# Linux/Mac
export DB_URL="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl"
export DB_USER="RMxxxxxx"
export DB_PASSWORD="..."
mvn quarkus:dev

# Windows (PowerShell)
$env:DB_URL="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl"
$env:DB_USER="RMxxxxxx"
$env:DB_PASSWORD="..."
mvn quarkus:dev
```

O servidor sobe em `http://localhost:8080`. Os endpoints ficam disponiveis sob o prefixo `/deepscan`.

5.6 Rodar via Docker (opcional)

```
docker build -t deepscan-java .
docker run -p 8080:8080 \
-e DB_URL="$DB_URL" \
-e DB_USER="$DB_USER" \
-e DB_PASSWORD="$DB_PASSWORD" \
```

6. Endpoints da API REST

URL base do deploy: <https://deepscan.labs-lcs-server.com/deepscan>

Em desenvolvimento local: <http://localhost:8080/deepscan>

Formato: application/json em todas as requisicoes/respostas.

6.1 Estacoes de Monitoramento

Verbo	URI	Descricao	Status esperado
GET	/estacoes	Lista todas as estacoes	200 OK
GET	/estacoes/{id}	Busca estacao por ID	200 / 404
POST	/estacoes	Cadastra nova estacao	201 / 400
PUT	/estacoes/{id}	Atualiza dados da estacao	200 / 400 / 404
DELETE	/estacoes/{id}	Remove estacao	204 / 404

6.2 Leituras de Telemetria

Verbo	URI	Descricao	Status esperado
GET	/leituras	Lista todas as leituras	200 OK
GET	/leituras/{id}	Busca leitura por ID	200 / 404
GET	/leituras/estacao/{id}	Lista leituras de uma estacao	200 / 400
POST	/leituras	Registra leitura e gera alerta	201 / 400
DELETE	/leituras/{id}	Remove leitura	204 / 404

6.3 Alertas

Verbo	URI	Descricao	Status esperado
GET	/alertas	Lista todos os alertas	200 OK
GET	/alertas/pendentes	Lista alertas nao resolvidos	200 OK
PUT	/alertas/{id}/resolver	Marca alerta como resolvido	200 / 404
DELETE	/alertas/{id}	Remove alerta	204 / 404

6.4 Especies Marinhas

Verbo	URI	Descricao	Status esperado
GET	/especies	Lista todas as especies	200 OK
GET	/especies/{id}	Busca especie por ID	200 / 404
POST	/especies	Cadastra nova especie	201 / 400
PUT	/especies/{id}	Atualiza dados da especie	200 / 400 / 404
DELETE	/especies/{id}	Remove especie	204 / 404

6.5 Avistamentos

Verbo	URI	Descricao	Status esperado
GET	/avistamentos	Lista todos os avistamentos	200 OK
GET	/avistamentos/{id}	Busca avistamento por ID	200 / 404
GET	/avistamentos/especie/{id}	Lista avistamentos por especie	200 / 400
POST	/avistamentos	Registra novo avistamento	201 / 400
PUT	/avistamentos/{id}	Atualiza avistamento	200 / 400 / 404
DELETE	/avistamentos/{id}	Remove avistamento	204 / 404

6.6 Zonas de Monitoramento

Verbo	URI	Descricao	Status esperado
GET	/zonas	Lista todas as zonas	200 OK
POST	/zonas	Cadastra nova zona	201 / 400
DELETE	/zonas/{id}	Remove zona	204 / 404

6.7 Exemplo de requisicao

```
POST /deepscan/leituras HTTP/1.1
Content-Type: application/json

{
  "idEstacao": 1,
  "horarioLeitura": "2026-06-10T12:00:00",
  "sst": 22.5,
  "waveHeight": 4.2,
  "wavePeriod": 11.0,
  "windSpeed": 85.0,
  "windDirection": 230.0,
  "earthquakeMagnitude": 6.1,
  "focalDepth": 25.0
}
```

Resposta esperada: **201 Created** com o objeto persistido (incluindo o `idLeitura` gerado). O `AlertaBO` dispara automaticamente um alerta nivel ALTO para essa leitura.

7. Regras de Negocio (Camada BO)

7.1 EstacaoMonitoraBO

- Nome da estacao e obrigatorio (nao pode ser vazio).
- Tipo deve ser BOIA, SATELITE ou SUBMARINA (case-insensitive, acentos removidos).
- Latitude entre -90 e 90. Longitude entre -180 e 180.
- Antes de atualizar ou deletar, verifica se o ID existe.

7.2 LeituraTelemetriaBO

- ID da estacao deve ser maior que zero.
- SST entre -5 e 50 graus Celsius. Altura de onda entre 0 e 50 metros.

- Magnitude sismica entre 0 e 10. Profundidade focal não pode ser negativa.
- Após persistir, chama `AlertaBO.gerarAlertaAutomatico()`.

7.3 AlertaBO — Classificação automática de risco

Toda leitura registrada gera um alerta automaticamente se atender um dos critérios:

Nível	Critério
ALTO	Magnitude sismica ≥ 5.5 OU onda $\geq 3.5\text{m}$ com vento $\geq 80\text{ km/h}$
MEDIO	Vento $\geq 60\text{ km/h}$ com SST $\geq 26^\circ\text{C}$ OU onda $\geq 2.5\text{m}$
BAIXO	Magnitude ≥ 3.0 OU onda $\geq 1.5\text{m}$
—	Abaixo de todos os limites: nenhum alerta gerado

O alerta é inserido no banco com status não resolvido. Operadores listam via `GET /alertas/pendentes` e marcam como resolvidos via `PUT /alertas/{id}/resolver`.

7.4 EspecieBO

- Nome científico e nome popular são obrigatórios.
- Status de conservação deve seguir o padrão IUCN: LC, NT, VU, EN ou CR.
- Habitat é obrigatório.

7.5 AvistamentoBO

- IDs de estação e espécie devem ser maiores que zero.
- Horário do avistamento é obrigatório.
- Quantidade avistada deve ser maior que zero.

7.6 ZonaMonitoraBO

- Nome da zona e país são obrigatórios.
- Antes de deletar, verifica se o ID existe.

8. Tratamento de Exceções

Todas as camadas lançam exceções customizadas. Os `@Provider` `ExceptionHandler` interceptam essas exceções e padronizam a resposta HTTP. Os recursos não têm `try/catch` — a lógica de mapeamento de erro fica centralizada nos mappers.

Exceção	Status HTTP	Quando ocorre
<code>DadoInvalidoException</code>	400 Bad Request	Validação de entrada falhou (BO)
<code>RecursoNaoEncontradoException</code>	404 Not Found	ID inexistente em GET, PUT ou DELETE
<code>PersistenciaException</code>	500 Internal Error	Falha no banco (<code>SQLException</code> ou driver)
Throwable (genérico)	500 Internal Error	Qualquer outra exceção não tratada

Formato da resposta de erro

```
{
  "erro": "mensagem descritiva do problema"
}
```

Para o caso `PersistenciaException` a mensagem real fica no log do servidor (`printStackTrace`) e o cliente recebe a mensagem generica "Erro interno no servidor. Tente novamente." — assim nao vazamos detalhes do banco.

9. Banco de Dados

9.1 Conexao

A classe `br.com.fiap.conexoes.ConexaoFactory` abre conexoes JDBC lendo URL, usuario e senha de variaveis de ambiente. Cada DAO abre sua propria conexao no metodo (sem pool), em um `try-with-resources`, garantindo fechamento ao final.

Parametro	Origem
Driver	<code>oracle.jdbc.driver.OracleDriver</code>
URL	Variavel <code>DB_URL</code> (ex.: <code>jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl</code>)
Usuario	Variavel <code>DB_USER</code>
Senha	Variavel <code>DB_PASSWORD</code>
Classe Java	<code>br.com.fiap.conexoes.ConexaoFactory</code>

9.2 Tabelas

Tabela	Conteudo
<code>ESTACAO_MONITORA</code>	Estacao de monitoramento (boia, satellite, submarina).
<code>LEITURA_TELEMETRIA</code>	Leituras periodicas de sensores oceanograficos/sismicos.
<code>ALERTA</code>	Alertas gerados automaticamente a partir das leituras.
<code>ESPECIE</code>	Catalogo de especies marinhas com classificacao IUCN.
<code>AVISTAMENTO</code>	Avistamentos de especies vinculados a uma estacao.
<code>ZONA_MONITORA</code>	Zonas geograficas de monitoramento.
<code>ESTACAO_ZONA</code>	Associativa N:M entre estacoes e zonas.

9.3 Geracao de IDs

Os IDs sao gerados pelo proprio DAO no momento do `INSERT` via `SELECT NVL(MAX(coluna), 0) + 1 FROM tabela` na mesma conexao. Essa estrategia evita dependencia de *sequences* Oracle, simplificando a portabilidade do script SQL.

9.4 Script DDL

O arquivo `src/main/resources/create_tables.sql` contem o `DROP/CREATE` de todas as tabelas mais inserts de dados de exemplo. Execute-o em sua ferramenta Oracle preferida antes de subir a aplicacao.

10. CORS

A classe `br.com.fiap.filter.CorsFilter` (anotada com `@Provider` e implementando `ContainerResponseFilter`) adiciona os cabecalhos CORS em toda resposta:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Headers: origin, content-type, accept, authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS, HEAD
Access-Control-Allow-Credentials: true
```

Isso permite que o frontend hospedado na Vercel consuma a API sem restricoes de same-origin.

11. Deploy

A aplicacao e empacotada como uma imagem Docker multi-stage e exposta publicamente via Cloudflare Tunnel. O pipeline esta definido em `.github/workflows/deploy.yml` e roda em um GitHub Actions runner self-hosted.

11.1 Pipeline

- Trigger: push em main ou workflow_dispatch.
- `docker build` com tags `:latest` e `:<sha>`.
- `docker stop / rm` do container antigo.
- `docker run` com `--restart always` e variaveis de ambiente injetadas a partir de GitHub Secrets.

11.2 Cadeia de exposicao publica

```
Cliente HTTPS
-> Cloudflare (TLS termination)
-> cloudflared tunnel
-> nginx (proxy por hostname)
-> Docker container Quarkus (porta 8080)
-> Oracle FIAP
```

11.3 Imagem Docker

O Dockerfile usa multi-stage para reduzir o tamanho final:

```
FROM maven:3.9-eclipse-temurin-17 AS build
...
RUN mvn -B -DskipTests package

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /workspace/target/quarkus-app/... ./
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "quarkus-run.jar"]
```

12. Links e Recursos

Recurso	URL
Deploy (Backend)	https://deepscan.labs-lcs-server.com/deepscan
Repositorio GitHub	https://github.com/deep-scan-fiap/DeepScan-Backend-Java
Frontend	https://deepscanfiap.vercel.app
Video Pitch	https://youtu.be/zb1og_9PUhc
Video Demonstracao	https://youtu.be/fbiX4byQEes

13. Apendice — Prints da Aplicacao

Os prints das telas do frontend e das chamadas dos endpoints estao anexados ao final deste documento.

Cada print mostra: (i) tela do frontend integrada com o backend, (ii) chamada HTTP real ao endpoint correspondente (Postman/Insomnia/devtools), (iii) resposta da API com status code e corpo JSON.